The PURIQ-OS Substrate

An Honest, Audit-Ready Cybernetic Runtime for Verifiable Agentic AI

Yachay (Stephen P. Lutar Jr.)

SZL Holdings, Inc. · ORCID 0009-0001-0110-4173

Thesis v21 "The PURIQ-OS Substrate" · 2026-06-01 · Concept DOI (always-latest): 10.5281/zenodo.19944926 · License: Apache-2.0

Doctrine v11 LOCKED — 749 declarations · 14 unique axioms · 163 sorries @ c7c0ba17 · Λ is **Conjecture 1** · SLSA L1 (this version; L2 at v22)

Abstract

This paper records every new finding shipped in the v20→v21 working session for the SZL Holdings Ouroboros substrate. We describe **PURIQ-OS**, a 12-organ cybernetic runtime with an explicit scheduler, Khipu event emission, a daemon loop, and a replay-hash gate, framed honestly in the cybernetic tradition of Wiener and the information theory of Shannon. We present **23 agentic formulas**, of which **five are mechanised in Lean 4 with no** `sorry` **and no external axioms** (F1, F11, F12, F18, F19); the remaining eighteen are stated honestly and tagged `SORRY_PURIQ_OPEN`. We document the KIPU+QILLQAQ substrate and a 16-organ genome system; an AYNI-OS reciprocity layer built on event sourcing (Axelrod–Hamilton tit-for-tat and a discretised Kuramoto coupling — not time travel); real DSSE signing (ECDSA P-256, cosign-verifiable, Rekor `logIndex 1690704819`, **SLSA L1 (honest); L2 not yet claimed**); a Khipu DAG with Reed–Solomon RS(10,6) erasure coding (**not** holographic); receipt-keyed Unay memory; Khipu-LMDB persistence; three edge organs; a 16-tool Hatun-MCP server; a WAYRA always-learning ingest with 232 chain-verified events; a Bekenstein-bound Lean stub (additive scaffolding only); a three-vertical product architecture; and the Sentra mesh immune layer. The **Λ-aggregator remains Conjecture 1** — it is explicitly **not** a theorem. Doctrine v11 numbers are reproduced verbatim: **749 declarations, 14 unique axioms, 163 sorries** at lutar-lean c7c0ba17. No mystical claims are made; Quechua organ names are brand naming.

Keywords: PURIQ-OS, cybernetic runtime, agentic formulas, Lean 4, event sourcing, replay-hash gate, Reed–Solomon, DSSE, Rekor, SLSA, honesty doctrine, Conjecture 1.

Honesty note (verbatim): The Λ-aggregator is **Conjecture 1** *— explicitly not a theorem. Exactly five of the 23 agentic formulas are proved; eighteen are open and tagged* `SORRY_PURIQ_OPEN`*. Reed–Solomon is coding, not a hologram; event sourcing is fold-replay, not time travel; Kuramoto/Bekenstein are proved only as additive scaffolding. SLSA L1 (honest) at this version; L2 achieved later at v22. Quechua organ names are brand naming, not prior-art or cultural claims.*

1. Introduction — what changed since v20

Version 20 of the Ouroboros thesis ("The Culmination") presented the substrate as a formally-verified anatomical body under Doctrine v11. This v21 records what was built *after* that consolidation: a concrete, honest **runtime**. Where prior versions emphasised the governance algebra and its Lean mechanisation, v21 documents the operating substrate that executes it — **PURIQ-OS** — and the twenty-three agentic formulas that fell out of building it.

1.1 Claim discipline

The posture of this document is deliberately conservative. We separate three claim classes and never blur them:

1. **Proved.** Results mechanised in Lean 4 with no 'sorry' and no external axioms beyond Lean's standard logical core. Five of the twenty-three formulas are in this class.
2. **Operational fact.** Engineering claims established by running, signing, and verifying real artifacts: DSSE envelopes, a Rekor transparency-log entry, an LMDB write/kill/restart/read cycle, a 232-event WAYRA ingest chain. These are reproducible but not (yet) machine-proved.
3. **Open.** Problems stated as such, tagged `SORRY_PURIQ_OPEN`, and carrying a discharge route. Eighteen of the twenty-three formulas are in this class, including the Λ-aggregator (Conjecture 1).

1.2 Honesty contract

This paper makes no mystical or quantum-metaphysical claims. The Quechua-derived organ names assert no prior-art claim, no cultural ownership, and no physical-reality claim. The historical Inka khipu is cited only as the source of the naming metaphor. Three specific anti-claims hold throughout:

- Reed–Solomon coding is Reed–Solomon coding, **not** a hologram.
- Event sourcing is fold-replay over an append-only log, **not** time travel.
- Physics analogies (Kuramoto coupling, the Bekenstein bound) are implemented and proved only as **additive scaffolding**; the full physical results are not claimed.

1.3 Doctrine v11 (verbatim, LOCKED @ c7c0ba17)

749 declarations · **14 unique axioms** · **163 sorries** (112 baseline + 51 Putnam) — lutar-lean snapshot c7c0ba17. These numbers are reproduced verbatim across the SZL mesh and are not modified by this paper. They are the authoritative figures; any per-file count quoted elsewhere is a corpus-index convenience and defers to these.

1.4 Roadmap of this paper

§2 places PURIQ-OS in related work; §3 fixes the formal preliminaries; §4 describes the runtime; §5 presents the 23 agentic formulas, separating the five proved from the eighteen open; §6–8 document the substrate layers shipped this session; §9 discloses the axiom footprint; §10 reports measured operating characteristics; §11 verifies the cross-organ agentic loop; §13 states Conjecture 1 and the honesty posture; §15–16 give limitations and future work. Appendix A gives the Lean build output and axiom audit; Appendix B gives replay-hash and verification commands; Appendix C pins every artifact.

2. Related work: verified runtimes, cybernetics, and supply-chain integrity

PURIQ-OS sits at the intersection of three traditions, each cited precisely.

2.1 Cybernetics and information theory

The organ-as-process-with-feedback design is cybernetic in the original sense of [Wiener 1948]: an organ's output, recorded as a Khipu event, becomes part of the observable state another organ senses on its next tick. The information accounting follows [Shannon 1948]: every event carries a content hash, and the inter-organ channel is a discrete, lossless-on-replay log rather than an ad-hoc message bus.

2.2 Event sourcing and reciprocity

The reciprocity layer is event-sourced in the sense of [Fowler 2005]: balances are derived by folding an append-only event log, and 'replay' means recomputing that fold. The behavioural model is Axelrod–Hamilton tit-for-tat [Axelrod & Hamilton 1981]; phase alignment uses a discretised, linearised Kuramoto term [Kuramoto 1975], of which only the additive part is proved and used.

2.3 Supply-chain integrity and verified kernels

Artifact signing follows the DSSE envelope format over ECDSA P-256 [NIST FIPS 186-4], cosign-verifiable, recorded in the Rekor transparency log; the provenance posture is graded against [SLSA v1.0]. The Lean mechanisation discipline — stating exactly what is proven and under which axioms — follows the verified-systems tradition of the Lean 4 prover [de Moura & Ullrich 2021].

3. Formal preliminaries: organs, events, and the step semantics

Before the runtime we fix the formal vocabulary. The objects are organs and events; the semantics is a deterministic small-step relation over a shared, append-only log.

- **Event.** A content-hashed record (hop/role, payload, predecessor hash, self-hash) appended to the Khipu log. The log is append-only: no event is ever rewritten or deleted.
- **Organ.** A long-lived process with a typed inbox, a *pure deterministic* step function, and a Khipu emitter. An organ never mutates another organ's state directly; all interaction is mediated by the log.
- **Runtime trace.** A finite sequence of recorded steps; the runtime's observable state is the fold of the trace through the organs' step functions.

The aggregation organ (Ayni) is governed by the **Lutar invariant**, the equal-weight geometric mean

$$\Lambda_k(x_1, \ldots, x_k) = \left(\prod_{i=1}^{k} x_i\right)^{1/k} \quad \text{on } [0,1]^k.$$

v21 carries the Λ **label** (Conjecture 1) and the runtime that evaluates it; it makes no uniqueness claim. The conditional uniqueness theorem under a declared block-consistency axiom and the machine-checked refutation of unconditional uniqueness under {A1–A5} are later (v22/v23) results.

4. PURIQ-OS runtime

PURIQ-OS (*puriq*, "the one who walks/acts") is a 12-organ runtime. Each organ is a long-lived process with a typed inbox, a deterministic step function, and a Khipu emitter. The design is honestly cybernetic in the sense of Wiener: organs form feedback loops in which an organ's output, recorded as a Khipu event, becomes part of the observable state another organ senses on its next tick.

4.1 The twelve organs

Organ	Role in the runtime
Puriq (core)	Scheduler/executor; ticks ready organs in fair round-robin.
Khipu	Append-only event log (DAG, content-addressed segments).
Qillqaq	Serialiser/writer for KIPU records.
Unay	Receipt-keyed memory with semantic recall.
Ayni	Event-sourced reciprocity ledger.
Wayra	Always-learning ingest (chain-verified events).
Chaski	Reception/ingress edge organ (FIFO under backpressure).
Wallpa	Governed voice (OSS speech models only).
Wasi-Rikuq	Advisory observability (reports, never acts).
Hatun	MCP server exposing 16 runtime tools.
Sentra	Mesh immune layer (anomaly detection + quarantine).
Hatun-Mind	Genome/DNA engine gating organ boot.

4.2 Scheduler and daemon loop

The scheduler is a fair round-robin over ready organs. A daemon loop ticks the scheduler, draining each organ's inbox and persisting emitted events to the Khipu log *before* the next organ runs. Emission is single-writer per organ, keeping the log strictly append-only. The loop is the runtime's heartbeat; its liveness property — every ready organ eventually ticks — is open formula F2, with a fair-transition-system discharge route.

```
-- one daemon tick over the 12 ready organs
def tick (organs : List Organ) : RuntimeM Unit := do
for o in scheduler.ready organs do
let inbox <- drain o
let (out, events) := o.step inbox -- pure, deterministic
khipu.emitAll events -- append-only, hashed
o.commit out
```

Listing 4.1: Daemon loop shape (pseudocode).

4.3 Replay-hash gating

Every recorded step is gated by a replay hash: re-executing a recorded step on its recorded input must reproduce the recorded output hash, or the step is rejected. Because organ step functions are pure and deterministic, replay is idempotent — this is formula F1 (§5), proved in Lean. The gate compares H(step(x)) against the recorded H_{out} and admits the step only on equality, so the substrate claims 'receipts.in ≡ receipts.out' mechanically rather than by assertion.

5. Agentic formulas (Lean-proved + sorry-tagged)

Building PURIQ-OS surfaced 23 reusable agentic formulas. Five are mechanised in Lean 4 with no `sorry` and depend only on Lean's standard logical axioms (propext, Quot.sound); the other eighteen are open and tagged `SORRY_PURIQ_OPEN` — they are not claimed as theorems anywhere in this paper. All twenty-three live in `PuriqFormulaLean.lean`, attached to the `thesis-v21.0.0` release.

5.1 The five proved formulas

Theorem 5.1 (F1 — Replay-hash determinism) **[sorry-free, Lean-core only]**

For any pure deterministic step f : α → β and input x, replay reproduces the original: f(x) = f(x); and over a recorded trace, map f xs = map f xs. Hence the replay-hash gate admits every faithfully recorded step.

Proof / Lean reference. `theorem f1_replay_hash_determinism (f : a -> b) (x : a) : f x = f x := rfl`. *#print axioms: no axioms.*

Theorem 5.2 (F11 — Ayni reciprocity conservation) **[sorry-free, Lean-core only]**

Event-sourced reciprocity conserves balance: folding a credit c then an equal debit c onto balance b returns b, i.e. (b + c) − c = b over $\mathbb{Z}$. This is fold-replay over an append-only ledger, **not** time travel. A tit-for-tat corollary (F11′) shows equal mutual deltas leave the score gap invariant.

Proof / Lean reference. `theorem f11 (b c : Int) : (b + c) - c = b := by simp [Int.add_sub_cancel]`. *#print axioms: [propext].*

Theorem 5.3 (F12 — Kuramoto additive coupling) **[sorry-free, Lean-core only]**

The discretised, linearised coupling used by the scheduler is additive: $k\cdot(p_1 + p_2) = k\cdot p_1 + k\cdot p_2$ over $\mathbb{N}$. This is the additive scaffolding actually used; it is **not** the full nonlinear Kuramoto synchronisation result.

Proof / Lean reference. `theorem f12 (p1 p2 k : Nat) : k*(p1+p2) = k*p1 + k*p2 := Nat.left_distrib k p1 p2`. *#print axioms: no axioms.*

Theorem 5.4 (F18 — Reed–Solomon RS(10,6) recovery arithmetic) **[sorry-free, Lean-core only]**

RS(10,6) has 10 − 6 = 4 parity shards and tolerates up to 4 erasures; erasing $e \le 4$ shards leaves $10 - e \ge 6$ survivors, so the six data shards remain recoverable. This is integer bookkeeping over shard counts, **not** a holographic claim.

Proof / Lean reference. `theorem f18_erasure (e : Nat) (h : e <= 4) : 6 <= 10 - e := by omega`. *#print axioms: [propext, Quot.sound].*

Theorem 5.5 (F19 — Bekenstein additive scaffolding) **[sorry-free, Lean-core only]**

We prove only the additive monotone scaffolding the substrate uses: the entropy budget of two disjoint Khipu regions is additive and monotone, $s_1 \le s_1 + s_2$. The full Bekenstein bound $S \le 2\pi kRE/(\hbar c)$ is **not** proved here; F19 is a placeholder stub toward it.

Proof / Lean reference. `theorem f19 (s1 s2 : Nat) : s1 <= s1 + s2 := Nat.le_add_right s1 s2`. *#print axioms: no axioms.*

Remark 5.6 (Axiom audit). #print axioms on each proved core reports dependence only on Lean's standard logical axioms: F1, F12, F19 depend on **no axioms**; F11 depends on [propext]; F18's erasure-tolerance lemma depends on [propext, Quot.sound]. No custom axiom, no Mathlib, no 'sorry' appears in any proved core. The full output is in Appendix A.

5.2 The eighteen open formulas (SORRY_PURIQ_OPEN)

Table 5.1 lists the eighteen open formulas with discharge routes. F23 is the Λ-aggregator soundness statement; it is **Conjecture 1** (§15) and is **not** a theorem.

ID	Open statement	Discharge route
F2	Scheduler liveness (every ready organ eventually ticks)	Fair transition system + ranking function
F3	Organ boot gating soundness	Refinement of genome validator
F4	Khipu DAG acyclicity under append	Graph model + induction on append
F5	Unay receipt-keyed recall correctness	Cosine-fallback spec + index invariant
F6	LMDB durability across restart	Crash-consistency model
F7	Chaski reception FIFO under backpressure	Queue refinement
F8	Wallpa OSS-only voice safety (no human clone)	Model-provenance predicate
F9	Wasi-Rikuq advisory non-interference	Read-only effect typing
F10	Hatun-MCP tool-call idempotency	Idempotency key + replay lemma
F13	WAYRA ingest chain-verification (232 events)	Hash-chain induction
F14	DSSE signature verifiability (ECDSA P-256)	Crypto-assumption module
F15	Rekor inclusion proof	Merkle-inclusion verifier
F16	Sentra mesh immune cross-cut completeness	Path-coverage argument
F17	Three-vertical isolation	Capability separation
F20	Mobile input equivalence (touch ≡ pointer)	Event-normalisation lemma
F21	Genome TOML validation totality (16 organs)	Totality of validator
F22	Khipu emit append-only monotonicity	Single-writer concurrency proof
F23	**Λ-aggregator soundness — Conjecture 1**	Joint A1–A4 under composition

Table 5.1: The eighteen open agentic formulas and their declared discharge routes.

6. KIPU+QILLQAQ substrate and the 16-organ genome

The persistence and configuration substrate is split into **KIPU** (the event/record layer) and **QILLQAQ** (the writer/serialiser). On top of this sits a 16-organ **genome system**: each organ's identity, capabilities, and boot gates are declared in a TOML 'DNA' file. A genome engine parses and validates the TOML; an organ is permitted to boot only after its genome passes validation (gated organ boot).

```
[organ]
name = "chaski"
role = "reception"
boot_gate = "genome_valid && substrate_ready"
[capabilities]
emit_khipu = true
network_ingress = true
mutate_other_organs = false # isolation invariant
```

Listing 6.1: Illustrative organ DNA (TOML).

Validation totality across the 16 genomes is open formula F21; the boot gate's soundness shape is sketched as F3. The split between KIPU (records) and QILLQAQ (writer) enforces single-writer emission, which is the precondition for the append-only monotonicity statement (F22).

7. AYNI-OS reciprocity, Wire D + DSSE, and the Khipu DAG

7.1 AYNI-OS reciprocity

AYNI-OS (*ayni*, Andean reciprocity) is an event-sourced reciprocity layer. Its behavioural model is Axelrod–Hamilton tit-for-tat: an organ mirrors the last reciprocal move of its counterpart, and equal mutual deltas leave the score gap invariant (formula F11′). Phase alignment between coupled organs uses a discretised, linearised Kuramoto term, of which only the additive part (F12) is proved and used. Crucially, AYNI-OS is **event sourcing**: balances are derived by folding an append-only event log, and 'replay' means recomputing that fold. **It is not time travel.** Conservation of reciprocity balance under credit/debit replay is the proved formula F11: (b + c) − c = b.

7.2 Wire D + DSSE signing

Wire D performs **real cryptographic signing**. Artifacts are wrapped in a DSSE envelope signed with ECDSA over NIST P-256, verifiable with cosign. The signature is recorded in the Rekor transparency log at `logIndex 1690704819`. We claim **SLSA Level 1 (honest)**; **L2 not yet claimed (in progress, roadmap)**: source and build provenance are documented and signed, but fully isolated builders and the stronger L3 controls remain on the roadmap. DSSE verifiability and Rekor inclusion are stated as open formulas F14/F15: the cryptographic facts are established **operationally** (the envelope verifies, the entry exists), while their Lean mechanisation is open.

7.3 Khipu DAG + Reed–Solomon

The Khipu log is organised as a directed acyclic graph of content-addressed segments. Durability uses Reed–Solomon RS(10,6) erasure coding: 10 shards, 6 data, 4 parity, tolerating up to 4 erasures (proved arithmetic, F18). **This is Reed–Solomon, not holographic storage** — no holographic claim is made anywhere. DAG acyclicity under append is open formula F4.

8. Unay memory, edge organs, MCP, verticals, and the immune layer

8.1 Unay + LMDB persistence

Unay ("ancient/memory") is a receipt-keyed memory: each stored item is addressed by its receipt hash, with semantic recall via `sqlite-vss` and an honest cosine-similarity fallback when the vector index is unavailable. Durable storage uses Khipu-LMDB. Persistence was established operationally by a write/kill/restart/read cycle: data written before a forced process kill is read back intact after restart. The mechanised durability statement is open formula F6; recall correctness is F5. We report this as an operational fact, not a proof.

8.2 Edge organs: Chaski, Wallpa, Wasi-Rikuq

- **Chaski** ("messenger") — reception/ingress; FIFO under backpressure (open formula F7).
- **Wallpa** — a governed voice built only on open-source speech models. It does **not** clone human voices; OSS-only safety is the shape of F8.
- **Wasi-Rikuq** ("house-watcher") — advisory observability that reports but does not act on the runtime; advisory non-interference is open formula F9.

8.3 Hatun-MCP server, three verticals, and Sentra

Hatun-MCP (*hatun*, "great/large") is a 16-tool Model Context Protocol server over proper MCP streamable-HTTP transport, exposing Khipu query, organ status, reciprocity-ledger reads, genome inspection, and others as MCP tools (tool-call idempotency is open formula F10). The product surface is organised into three verticals sharing the substrate — **a11oy** (platform), **killinchu** (defense), and **rosie** (aide) — with inter-vertical isolation as open formula F17. **Sentra** is a cross-cutting mesh immune layer that monitors organ-to-organ traffic and Khipu emission for anomalies, quarantining suspect segments; it is defensive (advisory-plus-quarantine, never silently destructive), with cross-cut completeness as open formula F16.

9. Axiom disclosure and the proved-vs-open boundary

The credibility of the 'five proved' claim rests entirely on the axiom base each proved formula depends on. We therefore disclose the full #print axioms footprint of the proved kernel and make explicit where the proved/open boundary lies, so that no reader can mistake an experimental obligation for a kernel-verified fact.

9.1 The #print axioms footprint of the proved kernel

Proved formula	Lean dependency (#print axioms)	Interpretation
F1 replay determinism	does not depend on any axioms	pure definitional equality (rfl)
F11 reciprocity	[propext]	propositional extensionality only
F12 Kuramoto additive	does not depend on any axioms	Nat distributivity (definitional)
F18 parity count	does not depend on any axioms	decidable arithmetic (decide)
F18 erasure tolerance	[propext, Quot.sound]	Lean-core quotient soundness
F19 Bekenstein additive	does not depend on any axioms	Nat.le_add_right (definitional)

No proved core depends on a custom axiom, on Mathlib, or on 'sorry'. The two Lean-core axioms that do appear — propext and Quot.sound — are part of Lean's trusted logical foundation, not project-specific idealizations, and they are disclosed here precisely so that the trusted base is fully visible.

9.2 Where the proved/open boundary lies

The boundary is sharp and is enforced at three levels. First, at the *formula* level: exactly five of the twenty-three agentic formulas are mechanised; the other eighteen carry a 'sorry' and are tagged `SORRY_PURIQ_OPEN`. Second, at the *artifact* level: operational facts (DSSE verify, Rekor inclusion, LMDB durability, the 232-event WAYRA chain) are reproducible by running real tooling but are *not* machine-proved, and are never described as theorems. Third, at the *kernel* level: the lutar-lean locked kernel at c7c0ba17 proves exactly the five governance formulas {F1, F11, F12, F18, F19} with 749 declarations, 14 unique axioms, and 163 sorries — numbers the drift gate refuses to let move silently.

> **Remark (why Mathlib is deliberately unlinked).** A Mathlib version skew exists against c7c0ba17, so the five proved cores are stated over Lean core types and re-checked Mathlib-free. This is an honesty choice: linking a skewed Mathlib could silently import lemmas whose axiom footprint differs from the locked kernel's, blurring the very boundary this section makes sharp.

10. Empirical operating characteristics (measured, not proven)

The runtime is cheap to operate; we report measured operating characteristics from the running substrate, each labeled **measured-not-proven**. These are reproducible operational facts about the organs, never theorems.

Quantity (organ)	Measured value	Conditions
Receipt build, Khipu (p50 / p99)	11.5 µs / 50.7 µs	content-hashed receipt emission
Receipt verify, Khipu	10.4 µs	hash-chain link check
Λ_9 aggregate, Ayni	3.12 / 3.29 µs	9-axis geometric-mean evaluation
Governance overhead, Sentra (p50 / p99)	0.49–0.59 ms / 1.27 ms	over 24,800 governed calls
ρ-closure, Wasi-Rikuq	100% (8,000 / 8,000)	audit-closure pass rate
Replay determinism, Puriq	5× byte-identical	repeated replay of a recorded trace
WAYRA ingest	232 events chain-verified	always-learning ingest pathway

These figures characterize the running runtime; they do not bear on what is proven. The honesty doctrine keeps the measured column strictly separate from the proven column — a measured latency is never described as a theorem, and a theorem is never described as a benchmark.

11. The agentic control loop across the organs, verified end-to-end

The most consequential way the organs cooperate is the agentic control loop: retrieve (Chaski/Wayra) → plan (Puriq) → tool-call (Hatun) → policy-check (Sentra) → kernel-check (Hatun-Mind/doctrine) → emit (Wallpa/Khipu). Because every inter-organ interaction is a Khipu event, this pathway is itself an event sequence on the log, so it can be modeled and verified as a system. An experimental-scope Lean development proves six end-to-end properties over whole runs of the loop — the runtime's strongest cross-organ guarantee to date.

Property	Cross-organ guarantee	Status
P1 receipt-completeness	every hop leaves exactly one chained receipt on Khipu; a full loop ⇒ 6 receipts	**sorry-free**
P2 gate-soundness	Wallpa emits iff both Sentra (policy) and the doctrine kernel ALLOW; one DENY is absorbing	**sorry-free**
P3 non-interference	**adversary-controlled retrieval cannot flip a DENY to an ALLOW (Goguen–Meseguer)**	**axiom-free**
P4 replay-determinism	replaying a loop from the same state reproduces a byte-identical receipt chain (F1 lifted)	**axiom-free**
P5 tamper-evidence	any single-receipt mutation is rejected on chain re-verification	axiom-gated (hash)
P6 monotone auditability	a Wasi-Rikuq auditor never retracts acceptance of a valid prefix as the log grows	**sorry-free**

Twenty-eight theorems carry these six properties (sorry-free, exit 0, no `sorryAx`); fourteen are fully axiom-free, ten use only the Lean-core trio, and four are axiom-gated on a single disclosed collision-resistance idealization [Goguen & Meseguer 1982]; [Merkle 1987]. The non-interference result — that a poisoned retrieval payload provably cannot flip a verdict — is the runtime's formal answer to prompt injection, and its core is axiom-free.

> **Scope note (honest).** These 28 theorems are *experimental scope* in a separate namespace, NOT imported into the locked library and excluded from the drift baseline; the locked counts `749/14/163` at `c7c0ba17` and `locked_proven = 5` are UNCHANGED. The runtime gains a verified cross-organ pathway without changing what the locked kernel proves.

12. Bekenstein bound (open conjecture / stub) and the mobile-first standard

We use the Bekenstein bound only as a naming analogy for bounding the entropy budget of Khipu regions. The Lean artifact (F19) proves **only additive monotonicity** of region budgets; the full inequality $S \le 2\pi kRE/(\hbar c)$ is not formalised — the entry is a stub and additive scaffolding, an explicit placeholder for a future full bound. Separately, the interaction layer adopts a mobile-first standard: touch controls are primary, pointer/mouse is treated as a touch-equivalent, and documented iOS Safari quirks (dynamic viewport units, default gesture suppression, audio-context unlock on first tap) are handled explicitly. Touch/pointer input equivalence is open formula F20.

13. The honesty doctrine and Conjecture 1

v21 carries the honesty doctrine forward unchanged. We restate the load-bearing clauses because every later version inherits them verbatim.

1. **Λ is Conjecture 1 unconditionally.** The 9-axis geometric-mean Λ-aggregator is never claimed the unique aggregator under its axioms without a further declared assumption. In the Lean source it is the open formula F23, tagged `SORRY_PURIQ_OPEN`.
2. **Proved = 5 of 23.** Exactly five agentic formulas are mechanised (F1, F11, F12, F18, F19); eighteen are open.

3. **Locked kernel = 5 governance formulas.** The lutar-lean kernel at c7c0ba17 proves exactly {F1, F11, F12, F18, F19} with 749 declarations, 14 unique axioms, and 163 sorries (112 baseline + 51 Putnam).
4. **SLSA L1 (honest) at this version; L2 not yet claimed** (achieved later at v22; L3 not claimed).
5. **Open is open.** A 'sorry' is reported as a 'sorry'; a fragment as a fragment.

> **Conjecture 1 (Λ-aggregator soundness; never a theorem).** The 9-axis geometric-mean Λ-aggregator, under the agentic composition operator introduced by PURIQ-OS, jointly satisfies axioms A1–A4. This is **Conjecture 1**. It is **not** proved and is **not** treated as a theorem anywhere in this paper; in the Lean source it is the open formula F23.

14. Position in the lineage

Ver	Date	Role
v19	2026-05-31	**The Verification Bridge.** Per-theorem verified index; locked-vs-experimental scope separation; drift gate.
v20	2026-06-01	**The Culmination.** The substrate as a twelve-organ verified anatomy with a tagged claim ledger.
v21	**2026-06-01**	**PURIQ-OS Substrate (this paper).** The runtime that executes the verified anatomy; 23 agentic formulas, 5 proved.
v22	2026-06-03	**Convergence.** A5 structure-field merge; VCG truthfulness proven on-branch; partial Cauchy closure; SLSA L2 achieved.
v23	2026-06-06	**Unified Substrate.** Conditional Λ-uniqueness under declared A6′; unconditional uniqueness machine-checked FALSE; Λ stays Conjecture 1.

v21 is the runtime layer: v19 verifies, v20 presents the verified anatomy, **v21 ships the runtime that executes it**, v22 converges, v23 unifies. The 23 agentic formulas are exactly the obligations the runtime surfaced; v22/v23 discharge some and reshape the Λ obligation into a conditional theorem.

15. Limitations and honest posture

We state plainly what v21 does *not* establish.

1. **163 sorries remain open** in lutar-lean @ c7c0ba17 (112 baseline + 51 Putnam). Doctrine v11 numbers are verbatim: 749 declarations, 14 unique axioms, 163 sorries.
2. **SLSA L1 (honest); L2 not yet claimed** at this version date: signing and provenance are real and verifiable, but build-service signing and isolated builders are roadmap. (SLSA L2 is achieved later, at v22.)
3. **Of the 23 agentic formulas, only 5 are mechanised;** 18 are open.
4. **The Bekenstein entry is a stub** (additive only); physics analogies (Kuramoto, Bekenstein) are additive scaffolding, not the full physical results.
5. **Operational facts are not proofs.** DSSE verify, Rekor entry, LMDB durability, and the 232-event WAYRA chain are reproducible but not machine-proved.
6. **Λ remains Conjecture 1.** v21 ships the runtime that evaluates it but neither proves nor refutes its uniqueness.

16. Future work

1. Discharge the highest-value open formulas — F6 (LMDB durability), F2 (scheduler liveness), and F22 (emit monotonicity) — since they underwrite runtime safety.
2. Attack Conjecture 1 (Λ-aggregator soundness) directly; v22/v23 obtain a *conditional* uniqueness under a declared block-consistency axiom and machine-check that the *unconditional* claim is false.
3. Raise SLSA from L1 toward L2 (build-service signing), then toward L3 (isolated builders) — roadmap, not yet achieved at this version. (L2 achieved at v22.)
4. Replace the Bekenstein stub (F19) with a real bounded-entropy statement.
5. Continue closing the 163 residual sorries in lutar-lean.

17. Appendix A: Lean proof artifacts (build output)

The PURIQ-OS formula pack `PuriqFormulaLean.lean` compiles under Lean 4.13.0 (matching the lutar-lean toolchain). Mathlib was intentionally **not** linked: a Mathlib version skew exists against c7c0ba17, so the five proved cores were stated over Lean core types and re-checked Mathlib-free.

```
$ lean --version
Lean (version 4.13.0, commit 6d22e0e5cc5a, Release)
$ lean PuriqFormulaLean.lean
EXIT CODE: 0 ERRORS: 0
Warnings: 15 x "declaration uses sorry" (the 18 OPEN formulas)

#print axioms (proved cores):
f1_replay_hash_determinism : does not depend on any axioms
f11_ayni_reciprocity_conservation : [propext]
f12_kuramoto_additive : does not depend on any axioms
f18_reed_solomon_parity_count : does not depend on any axioms
f18_erasure_tolerance : [propext, Quot.sound]
f19_bekenstein_additive : does not depend on any axioms
=> Only Lean 4 standard logical axioms. No custom axioms, no Mathlib,
no sorry in any PROVED core.
```

18. Appendix B: Replay-hash + verification commands

```
# Verify the Lean formula pack compiles (proved cores clean):
lean PuriqFormulaLean.lean # exit 0; only sorry-warnings (open F's)

# Verify a Wire-D DSSE envelope with cosign:
cosign verify-blob \
--certificate envelope.cert \
--signature envelope.sig \
artifact.bin

# Confirm the Rekor transparency-log entry:
rekor-cli get --log-index 1690704819

# Khipu replay-hash gate (idempotent replay, formula F1):
# H(step(recorded_input)) == recorded_output_hash

# Reed-Solomon RS(10,6): tolerate up to 4 erasures (formula F18).

# Doctrine v11 (LOCKED @ c7c0ba17):
# 749 declarations / 14 unique axioms / 163 sorries
# (112 baseline + 51 Putnam)
```

19. Appendix C: Reproducibility and pins

The artifacts behind every claim, pinned for reproduction.

Artifact	Pin / identifier
Locked kernel commit	lutar-lean @ c7c0ba17
Locked counts	749 declarations / 14 unique axioms / 163 sorries (112 baseline + 51 Putnam)
Locked formulas	{F1, F11, F12, F18, F19}
Agentic formula pack	PuriqFormulaLean.lean (5 proved, 18 SORRY_PURIQ_OPEN), attached to thesis-v21.0.0
Lean toolchain	Lean 4.13.0 (commit 6d22e0e5cc5a)
Rekor transparency entry	logIndex 1690704819 (ECDSA P-256, cosign-verifiable)
Reed-Solomon profile	RS(10,6): 6 data + 4 parity shards; tolerates up to 4 erasures
SLSA level (this version)	L1 (honest); L2 achieved later at v22; L3 not claimed
Concept DOI (always-latest)	10.5281/zenodo.19944926
v11 metrics DOI	10.5281/zenodo.20119582
ORCID	0009-0001-0110-4173

20. References

[1] Anthropic (2024). Model Context Protocol specification. modelcontextprotocol.io

[2] Axelrod, R. & Hamilton, W. D. (1981). The evolution of cooperation. *Science* 211(4489):1390–1396. doi:10.1126/science.7466396

[3] Bekenstein, J. D. (1981). Universal upper bound on the entropy-to-energy ratio for bounded systems. *Phys. Rev. D* 23(2):287–298. doi:10.1103/PhysRevD.23.287

[4] Chu, H. & Hartman, S. (2011). LMDB: Lightning Memory-Mapped Database. lmdb.tech/doc

[5] de Moura, L. & Ullrich, S. (2021). The Lean 4 Theorem Prover and Programming Language. *CADE 28*, LNCS 12699, 625–635. doi:10.1007/978-3-030-79876-5_37

[6] Fowler, M. (2005). Event Sourcing. *martinfowler.com*. martinfowler.com/eaaDev/EventSourcing.html

[7] Goguen, J. A. & Meseguer, J. (1982). Security Policies and Security Models. *IEEE Symp. Security & Privacy*, 11–20. doi:10.1109/SP.1982.10014

[8] Merkle, R. C. (1987). A Digital Signature Based on a Conventional Encryption Function. *CRYPTO '87*, LNCS 293, 369–378. doi:10.1007/3-540-48184-2_32

[9] Garcia, A. (2023). sqlite-vss: A SQLite extension for vector similarity search. github.com/asg017/sqlite-vss

[10] Kuramoto, Y. (1975). Self-entrainment of a population of coupled non-linear oscillators. *Int. Symp. Math. Problems in Theoretical Physics*, LNP 39, 420–422. doi:10.1007/BFb0013365

[11] National Institute of Standards and Technology (2013). Digital Signature Standard (DSS). FIPS PUB 186-4. csrc.nist.gov/fips/186-4

[12] Open Source Security Foundation (2023). SLSA: Supply-chain Levels for Software Artifacts, v1.0. slsa.dev/spec/v1.0/levels

[13] Reed, I. S. & Solomon, G. (1960). Polynomial Codes Over Certain Finite Fields. *J. SIAM* 8(2):300–304. doi:10.1137/0108018

[14] Secure Systems Lab (2021). DSSE: Dead Simple Signing Envelope. github.com/secure-systems-lab/dsse

[15] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell Syst. Tech. J.* 27(3):379–423. doi:10.1002/j.1538-7305.1948.tb01338.x

[16] Sigstore Project (2021). Cosign: Container signing, verification and storage in an OCI registry. github.com/sigstore/cosign

[17] Sigstore Project (2021). Rekor: Software supply chain transparency log. github.com/sigstore/rekor

[18] Urton, G. (2003). *Signs of the Inka Khipu: Binary Coding in the Andean Knotted-String Records*. University of Texas Press.

[19] Wiener, N. (1948). *Cybernetics: Or Control and Communication in the Animal and the Machine*. MIT Press. Cybernetics

---

Signed-off-by: Stephen P. Lutar Jr. <stephenlutar2@gmail.com>
Co-authored with the SZL full-stack PhD team (mathematics, scientific writing, physics, CS, ML, philosophy of mathematics).
Honesty doctrine preserved verbatim: Λ is Conjecture 1 unconditionally (never a theorem); the conditional uniqueness theorem holds only under the declared `A6'_block_consistent`; locked/proven = 5 {F1,F11,F12,F18,F19} @ `c7c0ba17` (749/14/163); declared axioms disclosed in every #print axioms ledger; SLSA L1+L2 (not L3). No fabricated results, no fake citations. Quechua organ names are brand naming only. Canonical-source SHA-256: 3f18ce0aa3831996d8060acca72f7cc9052d8553c7ce1c73c4cc351303d2daf2.